



به نام خدا

دانشکده‌ی مهندسی برق و کامپیوتر

یادگیری تعاملی - پاییز ۱۴۰۳ – ۱۴۰۴

تمرین سری دوم

فاطمه محمدی – ۸۱۰۱۹۹۴۸۹

سوال اول: ۲

سوال دوم: ٤

سوال سوم: ٥

بخش اول: (سوالات) ٥

سناریو اول: ٥

سناریو دوم: ٨

سناریو سوم: ١٠

بخش دوم: (پیاده‌سازی) ١٤

سوال چهارم: ٢٠

بخش اول: (سوالات) ٢٠

بخش دوم (پیاده‌سازی): ٢٣

منابع: ٢٩

سوال اول:

۱. **یادگیری نظارتی:**

- در این یادگیری، ، مدل با یک مجموعه داده برچسب دار (labeled dataset) اموزش داده میشود، به‌طوری‌که هر ورودی با خروجی صحیح مرتبط است.
- هدف مدل این است که رابطه‌ای بین ورودیها و خروجیها پیدا کند و خطای پیشبینی خود را با خروجیهای واقعی به حداقل برساند.
- این روش برای مسائلی که از قبل مجموعه داده با ورودی و خروجیهای مشخص وجود دارد، مناسب است.

۲. **یادگیری تقویتی:**

- در آموزش تقویتی، یک عامل (agent) با یک محیط (environment) با انجام اقدامات مختلف(Actions)، بازخوردهایی که به صورت پاداش یا جریمه دریافت میکند، در تعامل است.
- هدف عامل در این مدل این است که با یادگیری از این بازخوردها، اقدامات بهینه‌ای را انجام دهد که مجموع پاداشهای دریافتی را در طول زمان حداکثر کند.
- این روش زمانی مناسب است که راه حل بهینه از قبل مشخص نیست و باید با آزمون و خطا کشف شود.

- **مثال اول: بازی**

برای مثال، یک بازی داریم که در یک صفحه n×m تعریف شده است. هدف، رسیدن از یک موقعیت شروع (Start) به یک موقعیت نهایی (Goal)است که با رسیدن به آن، امتیاز +۱۰ به بازیکن داده میشود.

هدف ما پیدا کردن مسیر بهینه از خانه شروع به موقعیت نهایی است.

صفحه به‌صورت زیر تعریف می‌شود:

۱. بازیکن در خانه ی شروع (Start) قرار دارد.
۲. بازیکن میتواند در هر حرکت به یکی از چهار جهت (بالا، پایین، چپ، راست) حرکت کند.
۳. برخی خانه‌ها ممکن است دارای مانع (Obstacle) باشند (عبور از آنها ممکن نیست).
٤. خانه‌ای که هدف (Goal) است، امتیاز +۱۰ دارد.

- **یادگیری نظارتی:**

- در این حالت باید، از قبل مجموعه‌ای از مسیرهای بهینه که از موقعیتهای مختلف صفحه به هدف منجر میشوند، را داشته باشیم.
(هر داده شامل: وضعیت فعلی بازیکن و صفحه (ورودی) و حرکت بهینه(خروجی) میباشد.)
- مدل با یادگیری از مجموعه داده، بهترین حرکت برای هرموقعیت را یادمیگیرد.
- این روش وابسته به مسیرهای از پیش تعریف شده است و برای زمانی که محیط بسیار متغیر باشد، کارایی ندارد.

- **یادگیری تقویتی:**

- فرایند یادگیری به صورت ساده شده:
عامل :(Agent) بازیکنی که در صفحه حرکت می‌کند.
محیط :(Environment) صفحه n× m شامل موانع و هدف.
عملیات :(Actions) حرکت به بالا، پایین، چپ، یا راست.
پاداش:(Reward)
  - رسیدن به خانه نهایی: +۱۰
  - برخورد به مانع یا خروج از صفحه : -۱

    - حرکت به خانه های دیگر: ۰
- عامل با آزمون و خطا (exploration) در محیط حرکت میکند و با دریافت پاداش/جریمه، یاد میگیرد چگونه مسیر بهینه را پیدا کند.

- **مثال دوم: شناسایی اسپم بودن یا نبودن ایمیل**
    - **یادگیری نظارتی:**
- در این حالت باید، از قبل یک مجموعه داده شامل ایمیلهای برچسب دار (**Labelled Dataset**) که مشخص میکند هر ایمیل "اسپم" یا "غیر اسپم" است، داشته باشیم.
- مدل با یک **classifier** و با استفاده از مجوعه داده برچسب دار، اموزش میبیند.
- در این مدل هدف یادگیری رابطه میان ویژگی های ایمیل و برچسب(اسپم/غیراسپم) و **حداقل کردن خطا** است.
- پس از یادگیری مدل ایمیلهای جدید را براساس الگوهای یادگرفته شده دسته بندی میکند.

    - **یادگیری تقویتی:**

فرض کنید از قبل هیچ داده برچسب داری موجود نباشد و فرایند برچسب گذاری دشوار باشد.

- فرایند یادگیری به صورت ساده شده:

عامل :(Agent) یک فیلتر ایمیل که تصمیم میگیرد ایمیل را اسپم یا غیر اسپم علامت گذاری کند.
محیط :(Environment) سیستم ایمیل که شامل ایمیلهای دریافتی و بازخورد کاربران است.
عملیات :(Actions) برچسب‌گذاری یک ایمیل به‌عنوان "اسپم" یا "غیر اسپم".
پاداش:(Reward)

- اگر ایمیل به درستی به‌عنوان اسپم شناسایی شود: +۱
- اگر ایمیل به درستی به‌عنوان غیر اسپم شناسایی: +۱
- اگر اشتباه دسته‌بندی شود : -۱

- عامل با آزمون و خطا (exploration) ایمیلها را برچسب گذاری میکند و از بازخوردهای مثبت یا منفی (پاداش/جریمه) یاد میگیرد که چگونه تصمیم‌گیری بهتری انجام دهد.
- در اینجا هدف **حداکثر کردن پاداش دریافتی** است.

- به طور کلی برای مثال اول، **RL** میتواند مفید تر باشد (باتوجه به تنوعی که ممکن است داشته باشیم و اینکه محیط میتواند پیچیده تر باشد و کاملا شناخته شده نباشد و یا متغیر باشد.)، برای مثال دوم معمولا مجموعه داده از قبل وجود دارد و محیط شناخته شده است، استفاده از **SL** در صورت وجود داده از قبل برچسب گذاری شده، میتواند مناسب تر باشد.

سوال دوم:

تعاریف اولیه

$$R'(s,a) = \alpha R(s,a) + \beta \quad (A) \qquad \alpha \neq 0$$

$$V^{\pi}(s) = E_{\pi}\left[\sum_{t=0}^{H} \gamma^t R(s_t, a_t) \mid s_0 = s\right]$$

$$V'^{\pi}(s) = E_{\pi}\left[\sum_{t=0}^{H} \gamma^t R'(s_t, a_t) \mid s_0 = s\right] =$$

$$= E_{\pi}\left[\sum_{t=0}^{H} \gamma^t [R(s_t, a_t) + \beta] \mid s_0 = s\right] = E_{\pi}\left[\sum_{t=0}^{H} \gamma^t \alpha R(s_t, a_t) + \sum_{t=0}^{H} \gamma^t \beta\right]$$

$$= \underbrace{E_{\pi}\left[\alpha \left[\sum_{t=0}^{H} \gamma^t R(s_t, a_t) \mid s_0 = s\right]\right]}_{= V^{\pi}(s)} + \underbrace{E_{H}\left[\sum_{t=0}^{H} \gamma^t \beta\right]}_{= C \text{ متغیر ثابت}}$$

$$\rightarrow V'^{\pi}(s) = \alpha V^{\pi}(s) + C \quad (B)$$

$$\pi'^{*}(s) = \arg\max_{\pi} V'^{\pi}(s) \overset{(B)}{=} \arg\max_{\pi} [\alpha V^{\pi}(s) + C] \rightarrow$$

C متغیر ثابت است و تأثیری ندارد.

$$\rightarrow \pi'^{*}(s) = \arg\max_{\pi} \alpha V^{\pi}(s) \quad (C)$$

حالت 1: $\alpha > 0 \rightarrow \max_{\pi}(\alpha V^{\pi}(s)) \equiv \alpha \max_{\pi} V^{\pi}(s) \rightarrow \arg\max_{\pi}(\alpha V^{\pi}(s)) = \arg\max_{\pi}(V^{\pi}(s))$

(C) $\rightarrow \pi'^{*}(s) = \arg\max_{\pi} V^{\pi}(s)$, $\pi^{*}(s) = \arg\max_{\pi} V(s)$ $\rightarrow$ در حالتی که $\alpha > 0$ داریم.

پاسخ تشویقی گرفت ✓

حالت 2: $\alpha < 0 \rightarrow \max_{\pi}(\alpha V^{\pi}(s)) \equiv \alpha \min_{\pi} V^{\pi}(s) \rightarrow \arg\max_{\pi}(\alpha V^{\pi}(s)) = \arg\min_{\pi} V^{\pi}(s)$

(C) $\rightarrow \pi'^{*}(s) = \arg\min_{\pi} V^{\pi}(s)$, $\pi^{*}(s) = \arg\max_{\pi} V^{\pi}(s)$ $\rightarrow$ در حالتی که $\alpha < 0$ تفاوت می‌کنند ✗

جمع‌بندی: در حالتی که $\alpha > 0$، پاسخ تشویقی گرفت، در حالتی که $\alpha < 0$ باشد، پاسخ تشویقی نگرفت.

قسمت دوم:

در حالت Continuing و $H \rightarrow \infty$، همچنان رابطه گفته شده برای R' و R، V'^{π} و V^{π} برقرار است و داریم برای $\alpha > 0$ تفاوتی ایجاد نمی‌شود، برای $\alpha < 0$ پاسخ تشویقی نگرفت.

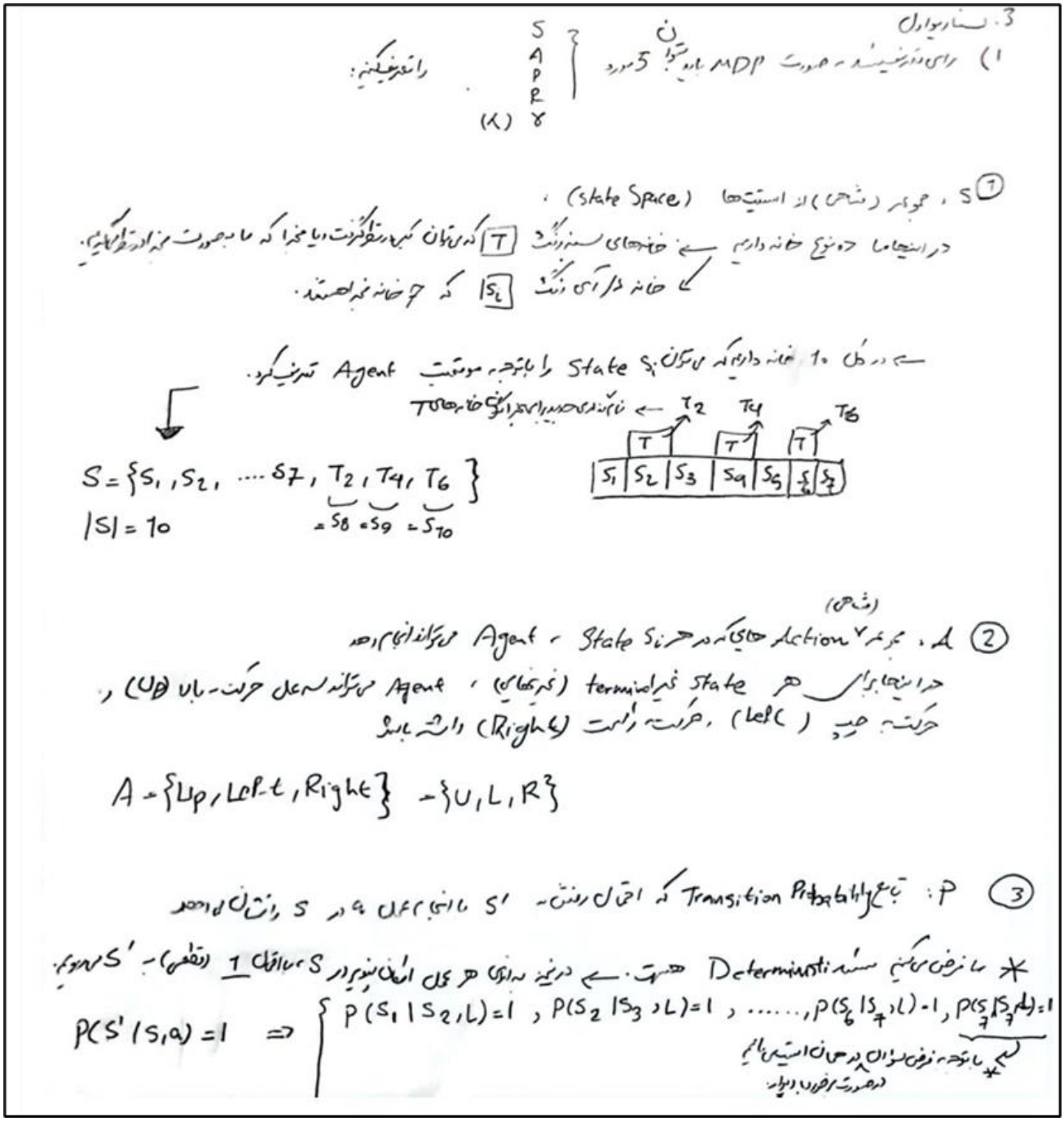
در حالت episodic و یا episodic task، چه در حالتی که Continuing task باشد چون تعامل روابط گفته شده برقرار نیستند. (در حالت episodic که قبلاً اثبات شد)

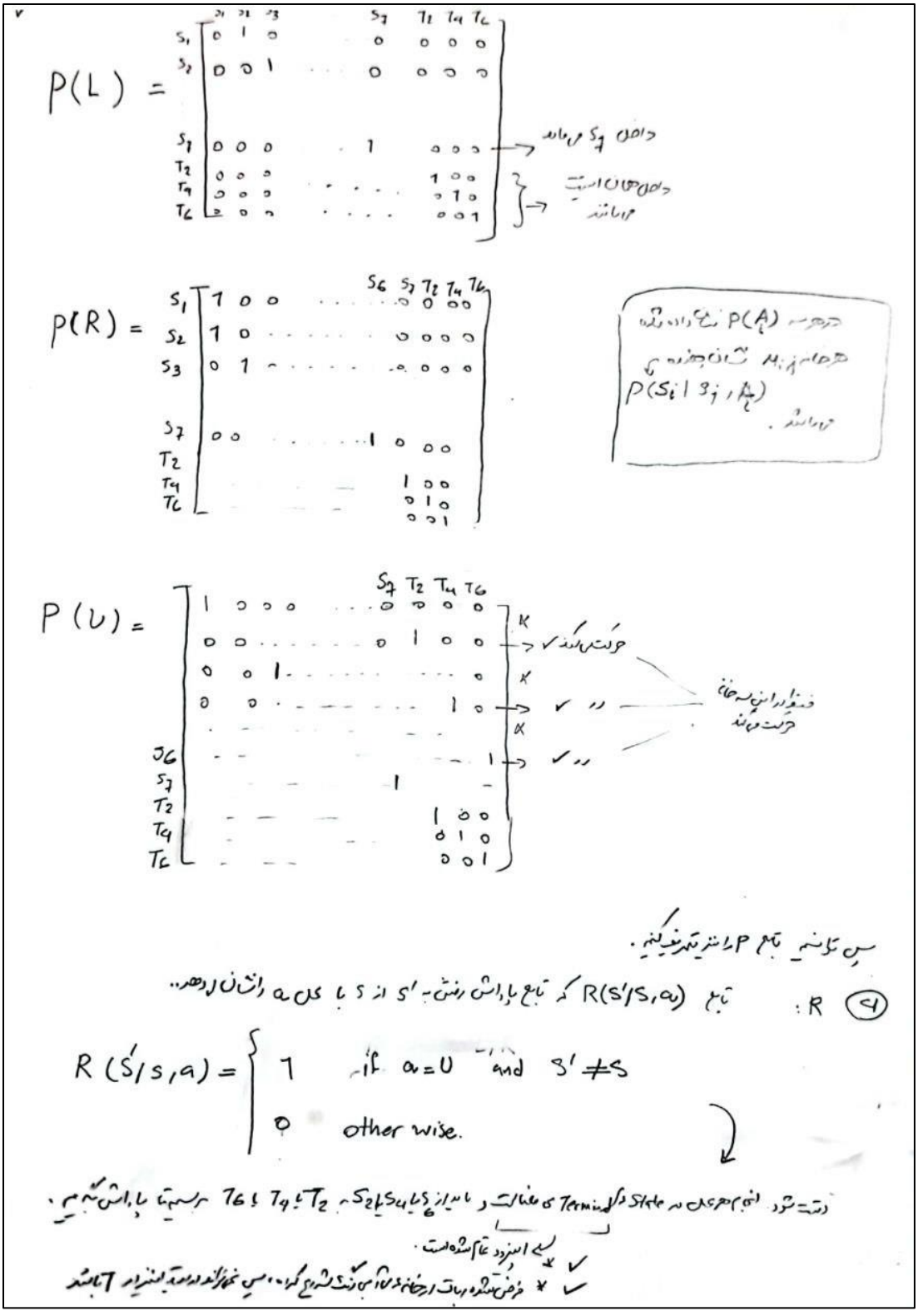
سوال سوم:

بخش اول: (سوالات)

سناریو اول:

الف)(۳ تصویر)





سپس تابع P را تعریف کنید.

(۴) R: تابع $R(S'|S,a)$ که تابع پاداش رفتن به S' از S با کنش a را نشان می‌دهد.

$$R(S'|S,a) = \begin{cases} 1 & \text{if } a=U \text{ and } S' \neq S \\ 0 & \text{otherwise.} \end{cases}$$

دقت شود اگر حرکت به خانه‌های Terminal State غیر از T_2 یا T_4 یا T_6 برسیم پاداش نمی‌گیریم.

✓ * فرض شده است.

✓ * فرض شده است ربات از خانه‌ی داده شده حرکت شروع کند، پس نمی‌گذارد وسط بیابان آب شود

(۵) در نهایت Discount Factor را (۰ یا ۱) را مانند تمرین کرد که عددی بین $[0,1]$ است در مسئله 0.9 استفاده شده است. ✓

۶-۵ ← توانستیم هر چنین MDP را تعریف کنیم ✓

ب)

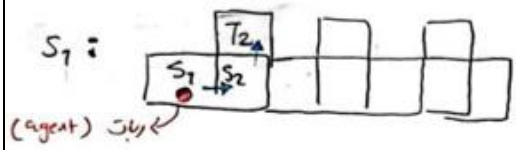
2)

برای هرکدام از $S_i \in \{S_1, \dots, S_7\}$ سیاست بهینه را بیابید. T_2 یا T_4 یا T_6 را نیز بیابید.

$T_2, T_4, T_6 \rightarrow$ State های خاتمه‌اند یعنی هیچ پاداشی بعد از آن‌ها نیست

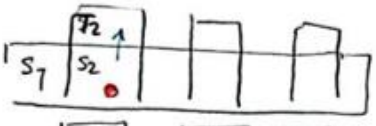
Agent ما ● سیرها ← → ↑

S_1 :

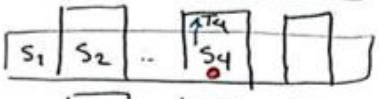


$\pi(S_1) = R$

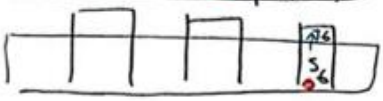
S_2 :



S_4 :



S_6 :



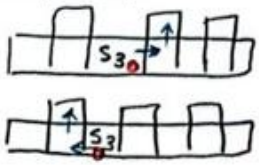
$\pi(S_2) = \pi(S_4) = \pi(S_6) = U$

S_7



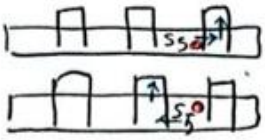
$\pi(S_7) = L$

$S_3 \rightarrow$ دو حالت دارد :



$\pi(S_3) = L$ or R

$S_5 \rightarrow$ دو حالت دارد



$\pi(S_5) = L$ or R

deterministic

پس در نهایت می‌توان $1 \times 1 \times 2 \times 1 \times 2 \times 1 \times 1 = 4$ یعنی ۴ سیاست بهینه داشت :

1) $\pi(S)$:
- $\pi(S_1) = R$
- $\pi(S_7) = L$
- $\pi(S_2) = \pi(S_4) = \pi(S_6) = U$
- $\pi(S_3) = L$
- $\pi(S_5) = L$

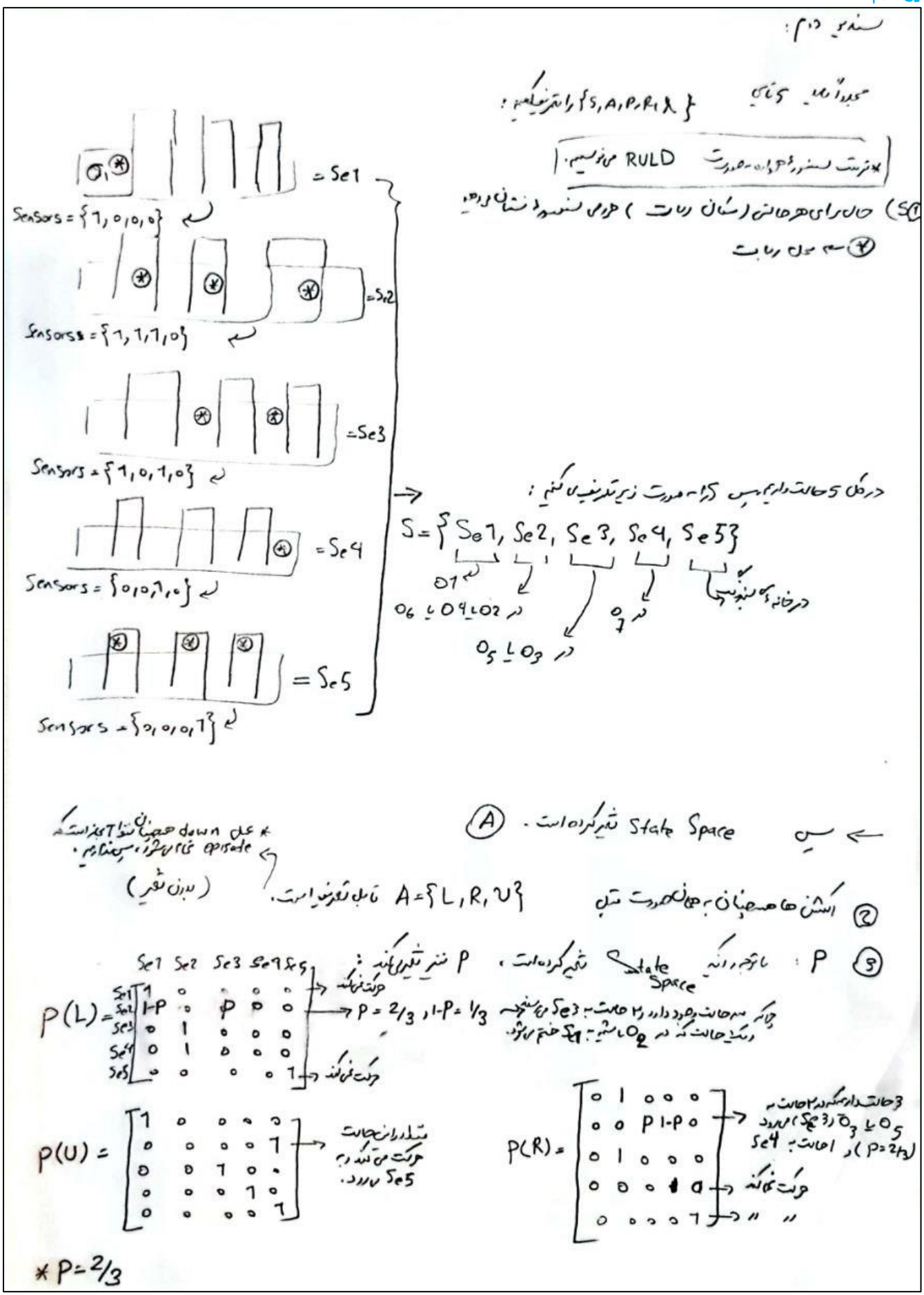
2)
- $\pi(S_1) = R$
- $\pi(S_7) = L$
- $\pi(S_2) = \pi(S_4) = \pi(S_6) = U$
- $\pi(S_3) = L$
- $\pi(S_5) = R$

3) $\pi(S)$:
- $\pi(S_1) = R$
- $\pi(S_7) = L$
- $\pi(S_2) = \pi(S_4) = \pi(S_6) = U$
- $\pi(S_3) = R$
- $\pi(S_5) = L$

4) $\pi(S)$:
- $\pi(S_1) = R$
- $\pi(S_7) = L$
- $\pi(S_2) = \pi(S_4) = \pi(S_7) = U$
- $\pi(S_3) = R$
- $\pi(S_5) = R$

برای T_2, T_4, T_6 چون Terminate می‌کنند، اجرای خود episode عامل‌شون دروغ!

سناریو دوم:


سناریو دوم:
مجدداً مانند گذشته {S, A, P, R, λ} را تعریف کنید:
* ترتیب حسگرها در وضعیت‌ها RULD می‌نویسیم.
(S) حالت‌های مختلف (مکان ربات) طوری که حسگرها نشان دهند
(A) محل ربات
Sensors = {1,0,0,0}
Sensors = {1,1,1,0}
Sensors = {1,0,1,0}
Sensors = {0,0,1,0}
Sensors = {0,0,0,1}
= Se1
= Se2
= Se3
= Se4
= Se5
در کل ۵ حالت داریم پس کلاً وضعیت زیر تعریف می‌کنیم:
S = {Se1, Se2, Se3, Se4, Se5}
O1
O6 تا O9 و O12
O7
O5 تا O3
در خانه‌ها که نمی‌توانیم
A. تغییر کرده است State Space
* علت down شدن ربات این است که
(بدون تغییر)
A = {L, R, U} تابع تعریف است.
② اکشن‌ها همانند حالت قبل
③ P: تابع رفتن، State Space تغییر کرده است، P تغییر می‌کند:
Se1 Se2 Se3 Se4 Se5
P(L) =
1 0 0 0 0
1-P 0 P 0 0
0 1 0 0 0
0 1 0 0 0
0 0 0 0 1
حرکت نمی‌کند
P = 2/3 , 1-P = 1/3
چرا که به احتمال P وارد Se3 می‌شود و به احتمال 1-P به O2 می‌رود و حالت که در Se1 ختم می‌شود.
حرکت نمی‌کند
P(U) =
1 0 0 0 0
0 0 0 0 1
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
متناظر با احتمالات حرکت می‌کند به Se5 می‌رود.
P(R) =
0 1 0 0 0
0 0 P 1-P 0
0 1 0 0 0
0 0 0 1 0
0 0 0 0 1
از حالت‌هایی که در احتمالات به O5 تا O3 (چپ) می‌رود Se4 با احتمال (P=2/3)
حرکت نمی‌کند
* P = 2/3

③ همانطور که مشاهده شد P تغییر کرده و از حالت Deterministic به Probabilistic درآمده است.

④ R تغییر می‌کند.

$$R\{Se' \mid Se, a\} = \begin{cases} 1 & \text{if } a = U \text{ and } Se' \neq Se \\ 0 & \text{otherwise.} \end{cases}$$

(درست است که تعریف پاداش تغییر نکرده است اما R در سناریوی قبل برای 70 حالت (70x3) تعریف می‌شود ولی در این سناریو برای 5 حالت (5x3) تعریف می‌شود)

⑤ γ تغییر نمی‌کند به همان صورت 0.9 = γ است

سناریو سوم: درصد کمی P و R تغییر کرده و از حالت Deterministic خارج شده‌ایم. ← A, B, C

سیاست بهینه:
- برای Se1 همان حرکت به راست هست — $\pi(Se1) = R$ (در O_1 قرار داریم)
- برای Se4 " " چپ هست — $\pi(Se4) = L$ (در O_7 هستیم)
- برای Se3 ← Episode تمام می‌شود. ×
- برای Se2 نیز همواره بالاترین حرکت به بالا - Se5 خواهیم رفت (در O_2 یا O_6 هستیم) $\pi(Se2) = U$
- برای Se3 نیز چه با حرکت - چپ یا راست به Se2 می‌رسیم و سپس با حرکت - بالا - خانه T خواهیم رفت $\pi(Se3) = L$ or R

به سیاست بهینه می‌توان به راحتی تعریف شود و دریافت حل می‌کند. (۲ سیاست بهینه داریم)

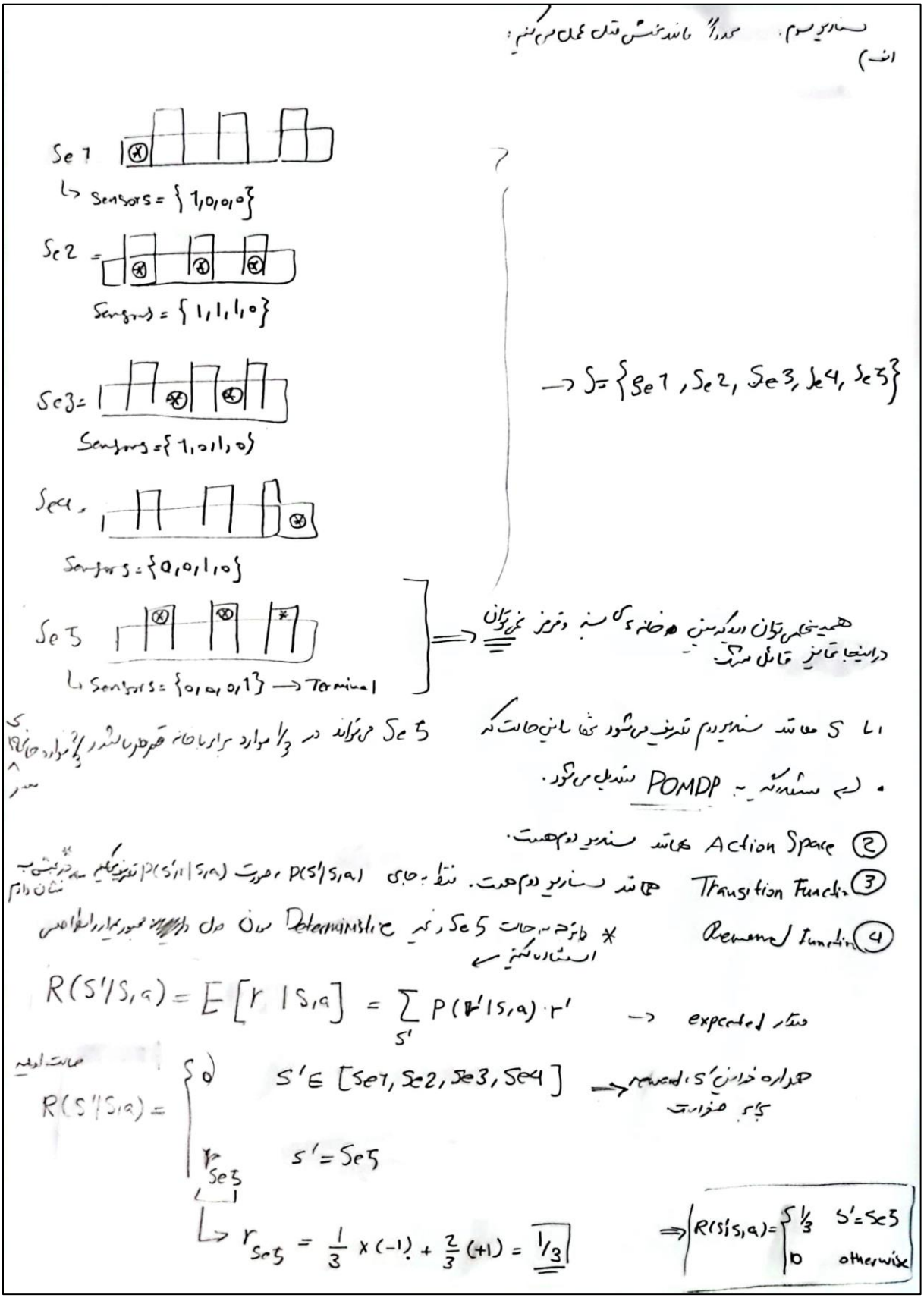
1) $\pi^*(S) = \begin{cases} \pi(Se1) = R \\ \pi(Se2) = U \\ \pi(Se3) = L \\ \pi(Se4) = L \end{cases}$

2) $\pi^*(S) = \begin{cases} \pi(Se1) = R \\ \pi(Se2) = U \\ \pi(Se3) = R \\ \pi(Se4) = L \end{cases}$

در Se5 ← Terminal می‌شود

سناریو سوم:

الف) (۲ تصویر)



← پس میتوانیم به صورت POMDP مدل سازی را انجام دهیم و هر 5 تایی $\{S, A, R, P, \gamma\}$ را تعریف کنیم

در اینجا چون بین خانه‌ی ترمز و بقیه خانه‌ها تفاوتی ایجاد کرد، ممکن است به Optimal Policy واقعی Coverage نکند - جایی که ما می‌خواهیم نرسیم.

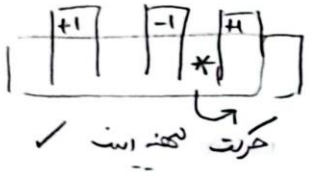
← حرکت‌های ناشی از تصادف شده می‌توان گفت Agent در Se5 - یک Policy - صورت زیر هنگرام می‌شود

$$\pi(S) = \begin{cases} \pi(Se1) = R \\ \pi(Se2) = U \\ \pi(Se3) = L \text{ or } R \\ \pi(Se4) = L \end{cases} \Rightarrow \boxed{-1}$$ باشد O_4

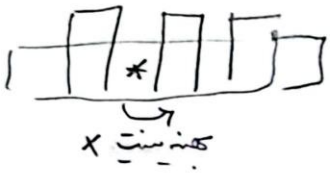
① در Se2 حرکت U انجام می‌شود که آنرا در O_4 می‌کنیم.

② اگر $\pi(Se3) = L$ یا $\pi(Se3) = R$ باشد هم مرتکب می‌شود در هردو در یک $\frac{1}{2}$ موارد سر به خانه‌ی خاص می‌شود.

$\pi(Se3) = L$

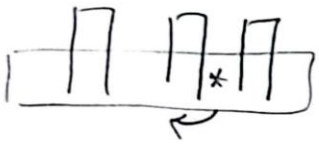


حرکت به خانه است ✓

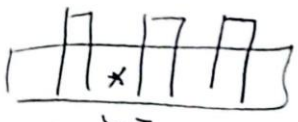


به خانه نیست ×

$\pi(Se3) = R$



به خانه نیست ×



به خانه است ✓

پس به دلیل ۲ است ← Se3 Se5 ← Policy کامل به خانه نمی‌رسیم. ← در Se5 همواره $\frac{1}{2}$ موارد و در Se3 همواره $\frac{1}{2}$ موارد رفتار غیر بهینه‌ای خواهیم داشت.

ب)

ب)

✓ Partialy observable (POMDP)
✓ Finite (FMDP)
✓ Stochastic
✓ Time-Invariant
✓ Reinforcement Learning
✓ Discrete
✓ 1 Agent

نوع MDP ؟

Single Agent_Finite_Stochastic_Time Invariant POMDP with Terminal States

$S = \{Se1, Se2, Se3, Se4, Se5\} = \{0,1,2,3,4\}$

* وضعیت سیاه: حرکت ربات امکان‌پذیر نیست

		Sensors	
0	Se1	1000	robot is in 01
1	Se2	0110	robot is in 02 or 04 or 06
2	Se3	1010	robot is in 03 or 05
3	Se4	0010	robot is in 07
4	Se5	0001	robot is in Red or Green Cells

where: 0 ... 4 = اندیس حالت‌ها

$A = \{R, U, L\} = \{0,1,2\}$

0	Right
1	UP
2	Left

$p(s', r \mid s, a) = ?$

$p(s', r \mid s, a) = p(s' \mid s, a) \cdot p(r \mid s', a, s)$

$p(s' \mid s, a) \Longrightarrow$ $p(a=0)$ (right) $=\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1-p & p & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$, $p(a=1)$ (up) $=\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$, $p(a=2) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1-p & 0 & p & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$

$p(0), p(1), p(2)$ احتمالات برای $a=0,1,2$ شانس رفتن.

* ماتریس‌های Matrix ماتریس‌های احتمالات در بیش‌تر مدل‌ها وجود دارند.

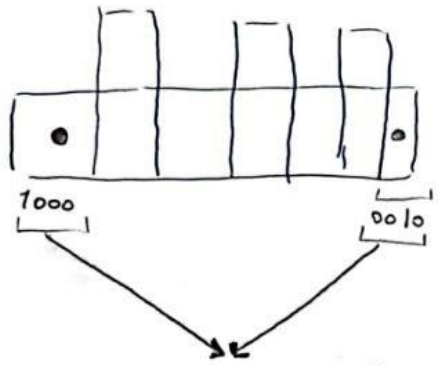
$p = \frac{2}{3}$

$$p(r \mid s', a, s) = \begin{cases} 1 & \text{if } r=0 \text{ and } s' \in [0,1,2,3] \\ 1/3 & \text{if } r=-1 \text{ and } s'=4 \\ 2/3 & \text{if } r=+1 \text{ and } s'=4 \\ 0 & \text{otherwise.} \end{cases}$$

} ⇒ خانه سبز یا قرمز

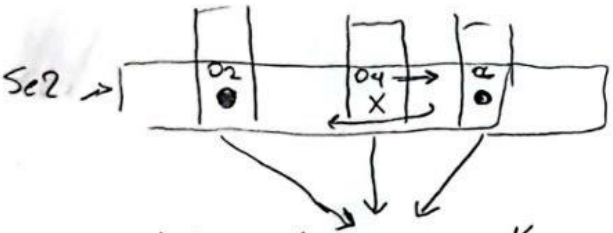
آیا می‌توان optimal Policy را به صورتی که ۱+ برابر پیدا کرد؟

(۱)



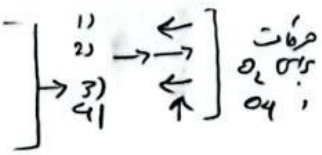
اگر Agent در این State ها بود کاملاً است به سمت Free (دیوار نیست) برود دستور بدهد بالا.

(۲)

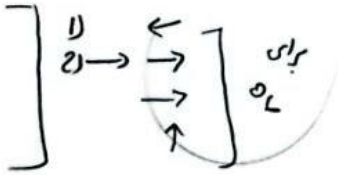


اگر Agent در اولین O_4 از این نقاط بود کاملاً است یکبار به چپ برود و دستور بدهد به راست برود تا سلول سمت چپ (دیوار سمت چپ) و دستور بدهد خانه راست (دیوار سمت راست) را متوجه کند.

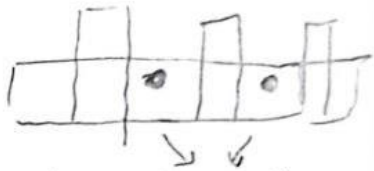
۱- اگر بر این نشانه‌ها یعنی یا O_2 یا در O_4 بوده است سپس سمت چپ برود تا به خانه اولیه برسد و بعد می‌رود بالا



۲- اگر مساوی با شد (منظور کسور ۱) یعنی در O_4 بوده است و الان در سمت راست آن یعنی در O_5 است کاملاً است یکبار برود به راست بعد دستور بدهد بالا برود.



(۳)



اگر در این خانه‌ها باشد کاملاً است ۲ بار به چپ برود (بدانت) اگر بعد از این حرکت در سمت چپ آن دیوار نباشد یعنی در O_3 بوده است، الان در O_1 حال کاملاً است یک بار به راست برود بعد ۲ بار بالا برود.

در غیر این صورت که دیوار در چپ در خود نقطه یعنی الان در O_6 است کاملاً است یکبار به چپ برود بعد دستور بالا.

(۴) Terminal است لازم به بررسی نیست

(۱)(۲)(۳)(۴) ←

توانستیم با چند حرکت امنیت برای هر State اطلاعات جدید پیدا کنیم که همین optimal policy را پیدا کنیم

- با داشتن ۱+ برابر شد ✓

بخش دوم: (پیاده‌سازی)

- فایل ipynb. ضمینه شده است.

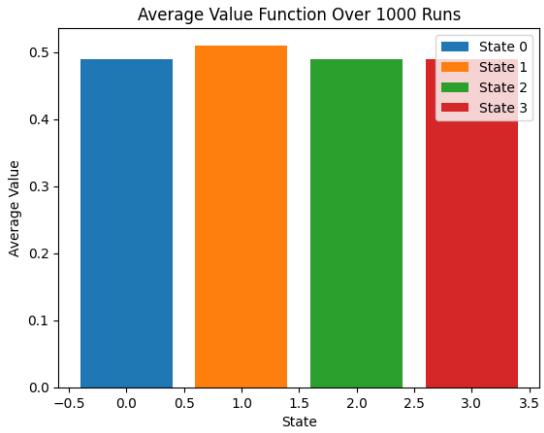
در اینجا فقط به بررسی نتایج میپردازیم:

توجه شود در PI برای انالیز بهتر و راحت تر (تکرار پذیری اجراها) از رندوم پالیسی اولیه استفاده نکرده ایم، با پالیسی اولیه حرکت به راست به ازای تمامی استیتها شروع کرده ایم.

الف)

در هیچ کدام از پیاده سازیهای انجام شده، به سیاست بهینه‌ای که در اخر منجر حتمی پاداش +۱ شود، نمیتوان دسترسی پیدا کرد و علت این مورد در بخش سوالات به طور کامل توضیح داده شده است، در برای بررسی الگوریتم‌ها، هرکدام را به تعداد مشخصی اجرا میکنیم و مقادیر **value_function** و **policy** نهایی در هر **iteration** را ذخیره میکنیم.

۱. پیاده سازی به کمک **Policy Iteration**



همانطور که مشاهده میشود، برای استیتها ۰و۲و۳ مقدار **Value function** کمتر و برابری داریم که مطابق انتظار است، چرا که به هیچ ریواردی ختم نمیشوند و شرایط یکسانی و برابر دارند. (به علت تقارن در مسئله و در خانه‌ها)
اما برای استیت ۱ که شامل بودن در خانه‌های **o2, o4, o6** میشوند، با توجه به اینکه به صورت میانگین میتوان با حرکت به بالا به پاداش $\frac{2*1-1*1}{3}$ که مقدار مثبتی است رسید، این استیت دارای **Value function** بیشتری نسبت به دیگر استیت ها میباشد.

```
policy_counts_PI_1 = Counter(tuple(policy) for policy in all_policies_PI_1)

unique_policies_PI_1 = [np.array(policy) for policy in policy_counts_PI_1.keys()]
repeat_counts_PI_1 = list(policy_counts_PI_1.values())

unique_policies_PI_1, repeat_counts_PI_1
```

✓ 0.0s Python

```
([array([0, 1, 0, 2, 0]), array([0, 0, 0, 0, 0]), array([1, 1, 1, 0, 0])],
 [544, 422, 34])
```

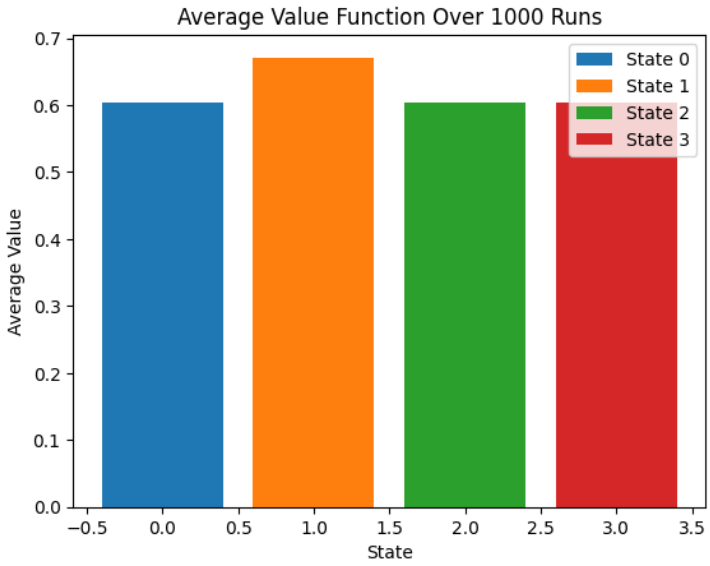
همانطور که در ابتدا گفته شد، نمیتوان به سیاست بهینه‌ای همواره همگرا شد و این مورد کاملا در بخش سوالات بررسی شده است، در نتیجه میبینیم که در این سناریو با کمک **PI** به ۳ سیاست متفاوت میرسیم که در اولی که پرتکرار ترین است درصورتی که ربات در **O1 – O3 -O5** شامل استیت **0,3** - باشد به سمت راست حرکت میکند و تنها زمانی که در **O7** – شامل استیت **4** - هست و راهی جز حرکت به چپ نیست، به چپ میرود، در حالتی که در **O2, O4**و **O6** باشد که استیت **1** میشود، بالا میرود که به امتیاز +۱ یا -۱ میرسد. چرا مقدار مورد انتظار برای این حرکت مقدار مثبت **+1/3** است. (استیت **4**، استیت ترمینال است و حرکت دیفالت "راست **0**" در نظر گرفته شده است و در همان استیت همواره میماند.)

در دومین پالیسی پرتکرار همواره به راست میرود، در واقع در این حالت ربات با امتیاز -۱ بیشتر روبرو شده است و ریسک گریز شده است در نتیجه در تمامی استیت‌ها حرکت به راست را انتخاب میکند که منجر میشود در نهایت با رسیدن ربات به خانه **O7** در همان نقطه همواره بماند.

اخرین پالیسی که تنها در این ازمایش ۳۴ بار انتفاق افتاده است، صرفا به دلیل وجود نداشتن پالیسی بهینه مشخص و ماهیت **Policy iteration** است که زمانی در خانه‌های **O2,O4,O6** باشد به بالا میرود و در غیر این صورت اکشن انتخابی که بالا یا راست است موجب باقی ماندن در مکانن قبلی میشود (به علت برخورد برخورد با دیوار).

*** دو دو پالیسی اول و سوم، ربات در صورت بودن در خانه‌هایِ پایینی خانه‌های ترمینال حرکت بالا انتخاب میکند، که حدودا دو سوم موارد را شامل میشود، که با توجه به اینکه در دو سوم موارد هم به پاداش +۱ میرسد از نظر اماری میتوان گفت نتیجه درستی است و در یک سوم موارد ربات با جریمه **-1** روبرو شده است و از رفتن به **Terminal** استیت‌ها خودداری میکند.

۲. پیاده سازی به کمک Value Iteration



نتایج تقریبا مطابق PI است با این تفاوت که تفاوت میان value_function استیت ۱ (o2,o4,o6) نسبت به باقی استیت ها به وضوح افزایش یافته است که در ادامه میبینیم به این دلیل است که تنوع سیاست‌های یافته شده در این روش کمتر است.

*** باقی جزییات که چرا برای استیت 1 مقدار value function بیشتر است و برای بقیه باهم برابر است، در قسمت قبل توضیح داده شده است.

```
policy_counts_VI_1 = Counter(tuple(policy) for policy in all_policies_VI_1)

unique_policies_VI_1 = [np.array(policy) for policy in policy_counts_VI_1.keys()]
repeat_counts_VI_1 = list(policy_counts_VI_1.values())

unique_policies_VI_1, repeat_counts_VI_1
```

✓ 0.0s Python

```
([array([0, 1, 0, 2, 0]), array([0, 0, 0, 0, 0])], [655, 345])
```

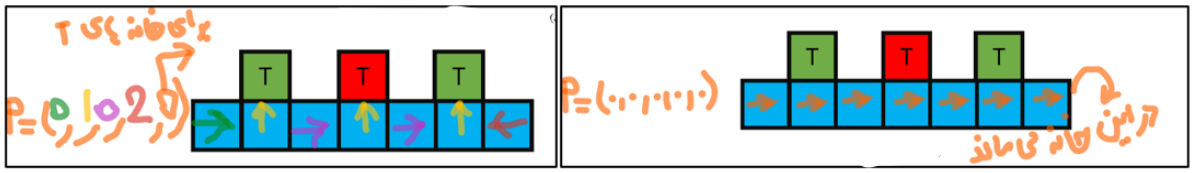
در اینجا مطابق قسمت قبل نیز به بیشتر از یک پالیسی میرسیم، که در همان پالیسی‌های پرتکرار در بخش قبل هستند و در قسمت قبل بررسی کرده‌ایم ربات با بودن در هر خانه با این دو پالیسی چه عملی را انجام میدهد.

تنها نکته قابل توجه این است در یک سوم موارد میتواند دید ربات از رفتن به خانه‌های ترمینال خودداری میکند که از نظر اماری نیز درست است و در یک سوم موارد ممکن است ربات با رفتن به خانه بالا امتیاز منفی یک بگیرد و از تکرار این عمل خودداری کند.

*** درواقع بین عمل 0 و 2 (رفتن به راست و چپ) در پالیسی دوم چه در این بخش چه بخش قبل (PI) تفاوتی وجود ندارد و صرفا چون اولین عمل است زمان انتخاب بهترین عمل که از argmax استفاده میکنیم، مقدار 0 که اولین عملی است که بیشترین مقدار را دارد برگردانده میشود و اگر ترتیب به صورت 2,1,0 بود یا رندوم بود، میتوانست به جای 0، 2 یعنی حرکت به چپ هم داشته باشیم.

*** برای خانه‌های ترمینال برای راحتی به جای None از مقدار دیفالت 0 استفاده شده است، که تفاوتی نمیکند و ایجنت در آن خانه میماند در هر صورت.

در نهایت حرکت ربات به ازای بودن در هرخانه، به ازای هرپالیسی را در دو تصویر زیر نشان داده ایم:



*** مقایسه کلی بین PI و VI:**

- خروجی این دو روش در قبل نشان داده شده و بررسی شده است و دیدیم تنها تفاوت جزئی وجود دارد که VI به صورت قطعی تر عمل میکند و همه سیاست‌ها را بررسی نمیکند (برخی از سیاستهای suboptimal در این روش در نظر گرفته نمیشوند. و تنوع کمتری دارد.)

مقایسه زمان:

به صورت خلاصه VI معمولا به دلیل سادگی و مقاومت خود، به‌ویژه درPOMDP های با عدم قطعیت زیاد، عملکرد بهتری دارد. که این مورد را در خروجی نمایش داده در ادامه نیز میتوان دید:

```
    execution_times_PI = []
    execution_times_VI = []

    for i in range(10):
        H = np.random.randint(100, 1000)
        execution_time_PI = timeit.timeit(lambda: policy_iteration(abstract_states, actions, abstract_transitions, abstract_reward), number=1)
        execution_time_VI = timeit.timeit(lambda: value_iteration(abstract_states, actions, abstract_transitions, abstract_reward), number=1)
        execution_times_PI.append(execution_time_PI)
        execution_times_VI.append(execution_time_VI)

    avg_execution_time_PI = sum(execution_times_PI) / len(execution_times_PI)
    avg_execution_time_VI = sum(execution_times_VI) / len(execution_times_VI)

    print(f"Avg Execution Time - PI: {avg_execution_time_PI:.5f} seconds")
    print(f"Avg Execution Time - VI: {avg_execution_time_VI:.5f} seconds")
```

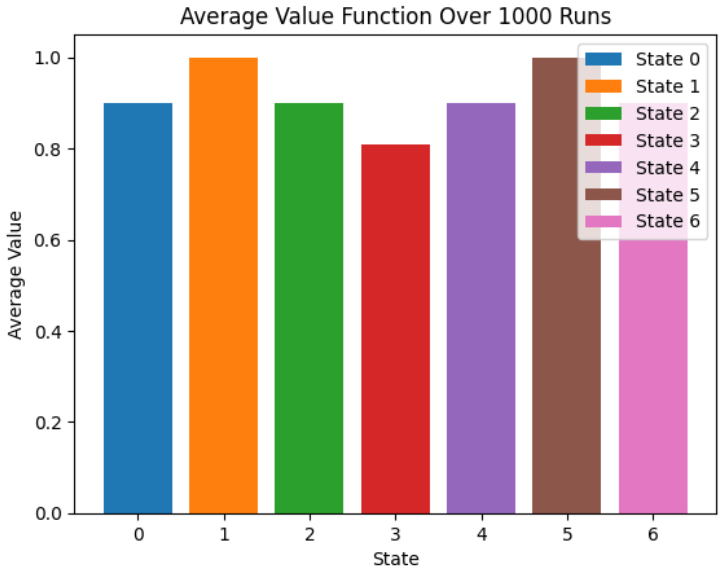
✓ 0.0s

```
Avg Execution Time - PI: 0.00037 seconds
Avg Execution Time - VI: 0.00028 seconds
```

ب)

در این حالت سیاست بیهنه وجود دارد (در بخش سوالات نشان داده‌ایم.) و ربات میتواند به آن برسد:

۱. پیاده سازی به کمک Policy Iteration



همانطور که مشاهده میشود، استیت ۳ (O4) کمترین مقدار را دارد که کاملا بدیهی است و به این علت است که این خانه در پایین خانه قرمز رنگ قرار دارد در نتیجه با حرکت به بالا ربات جریمه -۱ دریافت میکند. (نزدیک ترین خانه به خانه قرمز رنگ.)

در استیت های ۱ و ۵ که خانه‌های O2, O6 هستند بیشترین مقدار را داریم که به این علت است این دو خانه نزدیکترین خانه‌ها به خانه های سبز رنگ هستند و با حرکت به بالا میتواند در این خانه ها امتیاز +۱ دریافت کرد.

باقی خانه های نیز به علت تقارن مسئله و فاصله‌اشان از خانه‌های نهایی مقدار های یکسانی دارند،که کاملا توجیح پذیر است.

```
policy_counts_PI_2 = Counter(tuple(policy) for policy in all_policies_PI_2)

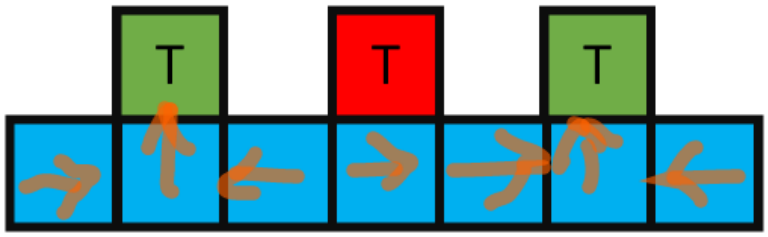
unique_policies_PI_2 = [np.array(policy) for policy in policy_counts_PI_2.keys()]
repeat_counts_PI_2 = list(policy_counts_PI_2.values())

unique_policies_PI_2, repeat_counts_PI_2
```

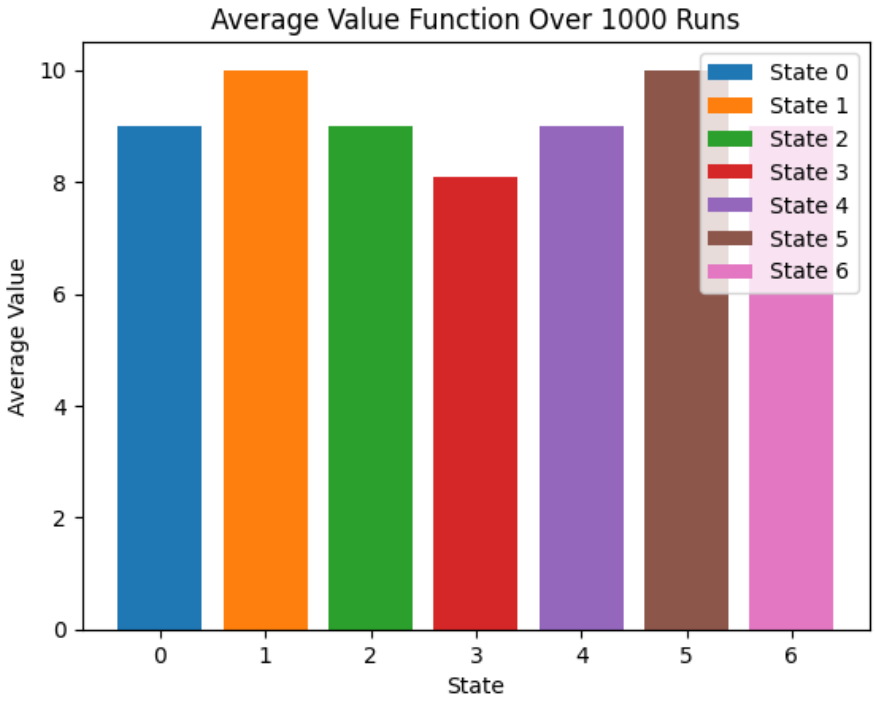
✓ 0.0s

```
([array([0, 1, 2, 0, 0, 1, 2, 0, 0, 0])], [1000])
```

میبینیم که در ۱۰۰۰ بار اجرا به سیاست بهینه رسیده‌ایم (شبیه ترین سیاست بهینه به سیاست بهینه اولیه – با توجه به اینکه از رندوم پالیسی استفاده نکرده ایم، در این صورت میتوانست به سیاست های متنوع ولی بیهنه برسد.) که همان چیزی است انتظار داشتیم و در بخش سوالات بررسی کرده‌ایم.



۲. پیاده سازی به کمک Value Iteration



کاملا به نتیجه یکسان با مرحله قبل (PI) رسیدیم، که به دلیل قطعی بودن و fully observable بودن محیط، انتظار میرفت.

** تحلیلهای این قسمت، مشابه قسمت قبل است پس از تکرار ان خودداری میکنیم.**

```
policy_counts_VI_2 = Counter(tuple(policy) for policy in all_policies_VI_2)

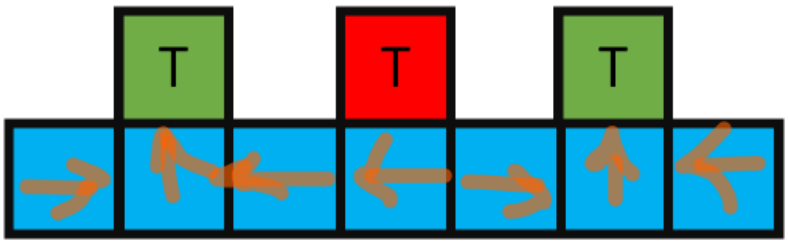
unique_policies_VI_2 = [np.array(policy) for policy in policy_counts_VI_2.keys()]
repeat_counts_VI_2 = list(policy_counts_VI_2.values())

unique_policies_VI_2, repeat_counts_VI_2
```

✓ 0.0s

```
[array([0, 1, 2, 2, 0, 1, 2, 0, 0, 0])], [1000])
```

مجددا میبینیم که در هر ۱۰۰۰ بار اجرا به یک جواب بیهنه (لزوما با PI یکسان نیست چرا که جوابهای بهینه متوعی داریم.) میرسیم که به دلیل قطعی بودن محیط در این حالت و اگاهی کامل ربات از محیط، همانچیزی است که انتظار داشتیم.



*** مقایسه کلی بین PI و VI:**

- خروجی این دو روش در قبل نشان داده شده و بررسی شده است و دیدیم هردو به جواب بهینه رسیدن، تنها تفاوت در سیاست نهایی بود که علت ان وجود چند سیاست بهینه است.

مقایسه زمان:

از نتایج نشان داده شده در ادامه، میتوان این نتیجه را گرفت که در این MDP قطعی، روش PI به‌طور قابل توجهی سریعتر از VIعمل کرده است.

دلیل این موضوع میتواند تعداد کمتر تکرارها یا کارایی محاسباتی بیشتر در به‌روزرسانی سیاستها در IP در مقایسه با به‌روزرسانی ارزشها در VI باشد.

```python
execution_times_PI = []
execution_times_VI = []

for i in range(10):
    H = np.random.randint(100, 1000)
    execution_time_PI = timeit.timeit(lambda: policy_iteration(states_in_real, actions_i
    execution_time_VI = timeit.timeit(lambda: value_iteration(states_in_real, actions_in
    execution_times_PI.append(execution_time_PI)
    execution_times_VI.append(execution_time_VI)

avg_execution_time_PI = sum(execution_times_PI) / len(execution_times_PI)
avg_execution_time_VI = sum(execution_times_VI) / len(execution_times_VI)

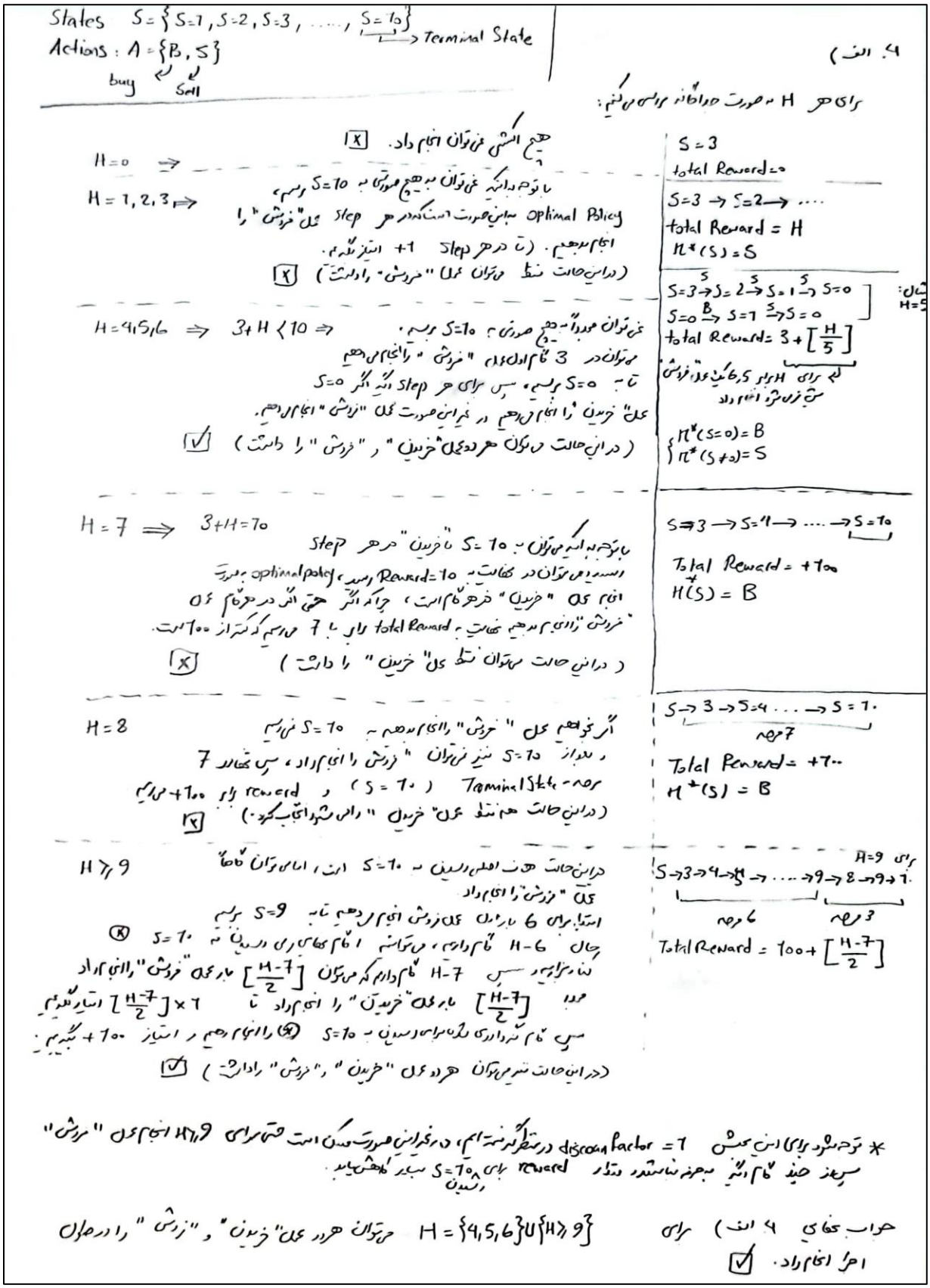
print(f"Avg Execution Time - PI: {avg_execution_time_PI:.5f} seconds")
print(f"Avg Execution Time - VI: {avg_execution_time_VI:.5f} seconds")
```

✓ 0.0s Python

```
Avg Execution Time - PI: 0.00019 seconds
Avg Execution Time - VI: 0.00483 seconds
```

سوال چهارم:

بخش اول: (سوالات)



* توجه شود برای این بخش discount Factor = 1 در نظر گرفته شده است، در غیر این صورت ممکن است هیچ‌گاه برای H≥9 حال انجام "فروش" بیشتر باشد.
سریع چند گام دیگر به جلو نشان داده و متوجه شویم reward برای S=10 بیشتر باشد.

جواب بخش ۴ الف) برای $H = \{4,5,6\} \cup \{H \geq 9\}$ می‌توان هردو حال "خرید" و "فروش" را در هر حالت انجام داد. √

ب ۴) ها نظر در مورد قبل بررسی کنیم برای $H=7,8,\dots$ حرزان است همانی $(S=10)$ دارید $(H>7)$

حواب کامل تر

* برای رسیدن به $S=10$ از $S=3$ باید $10-3$ بار عمل "خرید" انجام دهیم پس به حداقل 7 گام نیاز داریم

حواب همانی $\boxed{H\in[7,\infty)}$

ج ۴) $\lambda=0 \rightarrow G_t = r_t + \underbrace{r_t r + \dots}_{0} = r_t$ تنها immediate reward اهمیت دارد.

د ۱) $S=3$

$S=3 \begin{cases} S \ (\text{فروش}) \rightarrow \text{immediate reward} = +1 \\ B \ (\text{خرید}) \rightarrow \text{immediate reward} = 0 \end{cases} \rightarrow \boxed{\pi^*(S=3)=S}$

د ۲) $S=9$

$S=9 \begin{cases} S \ (\text{فروش}) \Rightarrow \text{immediate reward} = +1 \\ B \ (\text{خرید}) \Rightarrow \text{immediate reward} = +100 \end{cases} \rightarrow \boxed{\pi^*(S=9)=B}$

$\hookrightarrow \lambda=0 : \pi^*(S) = \begin{cases} \text{Sell} & \text{if } 0<S<9 \\ \text{Buy} & \text{if } S=0 \text{ or } S=9 \\ \text{Terminate} & \text{if } S=10 \end{cases}$

ج ۴) Discount factor : λ $\quad$ H is infinite

برای رسیدن به $S_{terminal}=10$ حداقل 7 گام از $S=3$ لازم است. (بعد گام خرید با پاداش صفر) $\left.\begin{matrix} S=3 \\ S_{terminal}=10 \end{matrix}\right\} \Rightarrow$

در این صورت - reward=+100 مرحله که در λ^6 ضرب می‌شود.

terminal state در گام $t+6$ $\quad$ Total reward $\boxed{R_b} = r_t + r_{t+1}\lambda + r_{t+2}\lambda^2 + r_{t+3}\lambda^3 + \dots + r_{t+6}$ $\quad$ Terminate در

$= 0 + 100\lambda^6 = 100\lambda^6$

برای $S=3,4,\dots,9$ $\quad$ $r_t = r_{t+1} = \dots = r_{t+5} = 0$ $\quad$ $r_{t+6} = 100$

$\hookrightarrow = 0$ $\quad$ $\hookrightarrow 100\lambda^6$

$\boxed{R_S} = \underbrace{r_t + r_{t+1}\lambda + r_{t+2}\lambda^2}_{\text{۳ بار پاداش "فروش" انجام می‌شود}} + \underbrace{0 + \lambda^4 + 0 + \lambda^6 + \dots}_{\text{صورت دنباله‌ی هندسی خرید و فروش}} = 1 + \lambda + \lambda^2 + \left(\frac{\lambda^4}{1-\lambda^2}\right)$

$r_t = r_{t+1} = r_{t+2} = 1$ به دلیل "فروش" $\quad$ $0 + \lambda^4 + 0 + \lambda^6 + \dots = \frac{\lambda^4}{1-\lambda^2}$ به دلیل خرید و فروش

حال برای اینکه $S=10$ سودآور باشد $R_S > R_B$ باشد $\leftarrow 1+\lambda+\lambda^2+\left(\frac{\lambda^4}{1-\lambda^2}\right) > 100\lambda^6$

به توجه - اینکه λ چه مقدار باشد - همواره می‌توان حرف تولید کرد

$1+\lambda+\lambda^2+\left(\frac{\lambda^4}{1-\lambda^2}\right) > 100\lambda^6$

$\frac{\lambda^4}{1-\lambda^2}$ در صورت حرف نزدیک به

$1-\lambda^2$ حرف ≈ 0

کدوش برقرارتر ☑

$1+\lambda+\lambda^2 > 100\lambda^6 \rightarrow 1+\lambda+\lambda^2-100\lambda^6 > 0 \rightarrow \lambda' = 0.570359$

$\lambda \in [0.577, 1]$ ←

نمودار

0.577

* در صورت حرف نزدیک به $\left(\frac{\lambda^4}{1-\lambda^2}\right)$:

$1+\lambda+\lambda^2+\frac{\lambda^4}{1-\lambda^2} > 100\lambda^6 \rightarrow (1+\lambda+\lambda^2)(1-\lambda^2)+\lambda^4 > 100\lambda^6(1-\lambda^2) \rightarrow 1+\lambda-\lambda^3 > 100\lambda^6-100\lambda^8$

$\lambda \neq 1$

حل در پایتون ←

نمودار

$1+\lambda-\lambda^3-100\lambda^6+100\lambda^8$:

0.336 0.984

$\rightarrow \lambda \in (0.336, 0.984)$

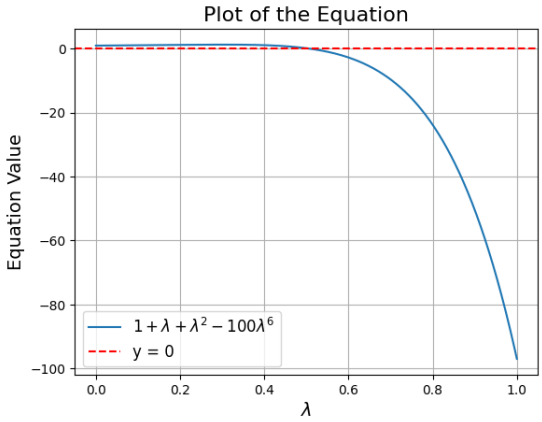
جوابی

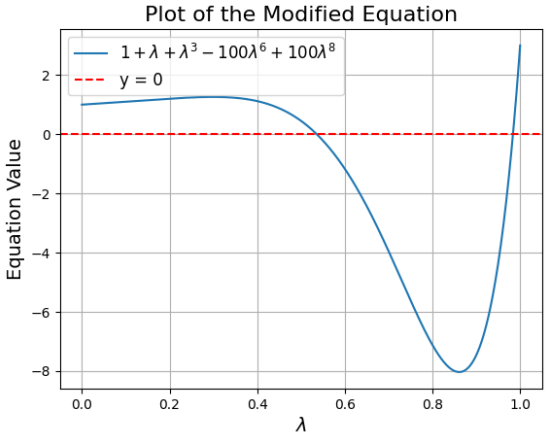
جواب درست ✓

S=10 یا S=3 از شروع optimal Policy ، بهتر $\lambda \in [0.336, 0.984]$ (در صورت جوابی) *

$$\pi^*(S) = \begin{cases} \text{Sell} & \text{if } S>0 \\ \text{Buy} & \text{if } S=0 \end{cases}$$

خواهد رسید و ← صورت دوره‌ای





بخش دوم (پیاده‌سازی):

- فایل ipynb. ضمینه شده است.

در ادامه کمتر به توضیحات کد و بیشتر به تحلیل نتایج خواهیم پرداخت:

الف)

توضیحات خلاصه پیاده سازی:

- در Finite horizon، برای انتخاب اکشن بهینه در هر "**زمان**" برای هر "**استیت**" باید هردو این عامل یعنی t و s را در نظر بگیریم، درواقع ارزش هراکشن دیگر فقط به استیت وابسته نیست و به زمان نیز وابستگی دارد. در واقع با توجه به اینکه چند step دیگر داریم، باید تصمیم گیری کنیم به این منظور از الگوریتم backward استفاده میکنیم (هم در PI و هم در VI)، درواقع این کار معادل این است که ربات پالیسی مناسب را برای h=0 و h=1و ... تا h=H بدست بیاورد و به این صورت میتواند در T=t ارزش انجام دادن هر اکشن و رفتن به استیت بعدی را با در نظر گرفتن ارزش استیت بعدی در زمانی که h=H-t که قبلتر کاوش کرده است(در این حالت با توجه به قطعی MDP بودن و fully observable بودن محیط نیاز به کاوش حتی نیست.) بدست بیاورد و به همین صورت برای هر استپ عمل میکند تا پالیسی بهنیه را برای هر استیت در همان استپ (t) بدست اورد. (جزییات پیاده‌سازی در فایل ضمینه شده قرار دارد.)

** با توجه به finite بودن افق گاما را برابر با ۱ در نظر میگیریم.

۱. پیاده سازی به کمک Policy Iteration

با توجه به توضیحات در بخش سوالات، برای H = 3, 5, 8, 9, 12 الگوریتم را اجرا میکنیم و دقیقا نتایج مطابق با نتایج گفته شده در بخش سوالات است:

```
PI:
H = 3,  #Sell = 3,  #Buy = 0,   -> Sell-only policy
Time Step: 0, State: 3, Action: Sell
Time Step: 1, State: 2, Action: Sell
Time Step: 2, State: 1, Action: Sell

H = 5,  #Sell = 4,  #Buy = 1,   -> Mixed policy (buy and sell)
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Sell
Time Step: 2, State: 3, Action: Sell
Time Step: 3, State: 2, Action: Sell
Time Step: 4, State: 1, Action: Sell

H = 7,  #Sell = 0,  #Buy = 7,   -> Buy-only policy
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Buy

H = 8,  #Sell = 0,  #Buy = 7,   -> Buy-only policy
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Buy
Time Step: 7, State: 10, Action: None
```

```
H = 9,  #Sell = 1,  #Buy = 8,   -> Mixed policy (buy and sell)
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Sell
Time Step: 7, State: 8, Action: Buy
Time Step: 8, State: 9, Action: Buy

H = 12, #Sell = 2,  #Buy = 9,   -> Mixed policy (buy and sell)
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Sell
Time Step: 7, State: 8, Action: Buy
Time Step: 8, State: 9, Action: Sell
Time Step: 9, State: 8, Action: Buy
Time Step: 10, State: 9, Action: Buy
Time Step: 11, State: 10, Action: None
```

۲. پیاده سازی به کمک Value Iteration

در این حالت نیز نتایج دقیقا مطابق انتظار مانند بخش قبل است:

```
VI:
H = 3,  #Sell = 3,  #Buy = 0,   -> Sell-only policy
Time Step: 0, State: 3, Action: Sell
Time Step: 1, State: 2, Action: Sell
Time Step: 2, State: 1, Action: Sell

_____________________________
H = 5,  #Sell = 4,  #Buy = 1,   -> Mixed policy (buy and sell)
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Sell
Time Step: 2, State: 3, Action: Sell
Time Step: 3, State: 2, Action: Sell
Time Step: 4, State: 1, Action: Sell

_____________________________
H = 7,  #Sell = 0,  #Buy = 7,   -> Buy-only policy
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Buy

_____________________________
H = 8,  #Sell = 0,  #Buy = 7,   -> Buy-only policy
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Buy
Time Step: 7, State: 10, Action: None
```

```
_____________________________
H = 9,  #Sell = 1,  #Buy = 8,   -> Mixed policy (buy and sell)
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Sell
Time Step: 7, State: 8, Action: Buy
Time Step: 8, State: 9, Action: Buy

_____________________________
H = 12, #Sell = 2,  #Buy = 9,   -> Mixed policy (buy and sell)
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Sell
Time Step: 7, State: 8, Action: Buy
Time Step: 8, State: 9, Action: Sell
Time Step: 9, State: 8, Action: Buy
Time Step: 10, State: 9, Action: Buy
Time Step: 11, State: 10, Action: None
_____________________________
```

- **مقاسه PI و VI:**

هردو الگوریتم را برای یک H رندوم اجرا میکنیم و طبق انتظار هردو به جواب بهینه و باتعداد برابر عمل "خریدن" و "فروش" میرسند:

Compare PI & VI:

+ Code + Markdown

```
Hs = [np.random.randint(20)]
start = np.random.randint(10)
print("PI:")
simulate_and_categorize(Hs, start_state, states, actions, transitions, rewards, finite_horizon_value_iteration)
print("VI:")
simulate_and_categorize(Hs, start_state, states, actions, transitions, rewards, finite_horizon_value_iteration)
```

✓ 0.0s

```
PI:
H = 12, #Sell = 2,      #Buy = 9,       -> Mixed policy (buy and sell)
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Sell
Time Step: 7, State: 8, Action: Buy
Time Step: 8, State: 9, Action: Sell
Time Step: 9, State: 8, Action: Buy
Time Step: 10, State: 9, Action: Buy
Time Step: 11, State: 10, Action: None
_____________________________
VI:
H = 12, #Sell = 2,      #Buy = 9,       -> Mixed policy (buy and sell)
Time Step: 0, State: 3, Action: Buy
Time Step: 1, State: 4, Action: Buy
Time Step: 2, State: 5, Action: Buy
Time Step: 3, State: 6, Action: Buy
Time Step: 4, State: 7, Action: Buy
Time Step: 5, State: 8, Action: Buy
Time Step: 6, State: 9, Action: Sell
Time Step: 7, State: 8, Action: Buy
Time Step: 8, State: 9, Action: Sell
Time Step: 9, State: 8, Action: Buy
Time Step: 10, State: 9, Action: Buy
Time Step: 11, State: 10, Action: None
_____________________________
```

مقایسه زمان اجرا:

اگرچه PI معمولا همگرایی سریعتری دارد، اما هزینه محاسباتی ان برای هر تکرار بیشتر است، به خصوص در این مسئله خاص که با توجه توابع نوشته شده میتوان دید که در VI ما یک حلقه for کلی د داریم، اما در PI تعداد حلقه‌های تودرتو بیشتر است.

```
execution_times_PI = []
execution_times_VI = []

for i in range(10):
    H = np.random.randint(100, 1000)
    execution_time_PI = timeit.timeit(lambda: finite_horizon_policy_iteration(states, actions, H, transitions, rewards), nu
    execution_time_VI = timeit.timeit(lambda: finite_horizon_value_iteration(states, actions, H, transitions, rewards), num
    execution_times_PI.append(execution_time_PI)
    execution_times_VI.append(execution_time_VI)

avg_execution_time_PI = sum(execution_times_PI) / len(execution_times_PI)
avg_execution_time_VI = sum(execution_times_VI) / len(execution_times_VI)

print(f"Avg Execution Time - PI: {avg_execution_time_PI:.5f} seconds")
print(f"Avg Execution Time - VI: {avg_execution_time_VI:.5f} seconds")
```

✓ 0.8s

```
Avg Execution Time - PI: 0.08213 seconds
Avg Execution Time - VI: 0.00696 seconds
```

ب)

در حالت infinite horizon داستن زمان کمکی نمیکند و در واقع دیگر مثل قبل نیاز به در نظر گرفتن زمان برای بدست اوردن پالیسی مناسب هر استیت نداریم، اما در عوض باید یک epsilon تعین کنیم تا از حلقه بینهایت در محاسبه پالیسی جلوگیری شود و اگر تغییرات value برای اکشنهایی که داریم از این اپسیلون کمتر بود دیگر به اپدیت کردن و بهبود پالیسی نپردازیم.

۱. پیاده سازی به کمک Policy Iteration

در حالتی که گاما برابر با صفر است ما تنها به پاداش در لحظه اهمیت میدهیم، در نتیجه به صورت بلند مدت برنامه ریزی انجام نمیشود و در این مسئله این عمر باعث میشود حتی با وجود اینکه میتواند با ۷ گام (با شروع از s=3) به پاداش +۱۰۰ رسید، ایجنت به امتیاز های +۱ در لحظه فقط اهمیت میدهد، پس در هر استیت (به غیر از ۹ و ۰) عمل "فروش" که پاداش یک را دارد انتخاب میکند، اما در استیت 0 نمیتواند هیچ فروشی داشته باشد درنتیجه مجبور به خرید است و در استیت ۹ تنها استیتی است که امتیاز فروش کمتر خرید است، عمل خرید را انجام میدهد.

در خروجی زیر نیز میتوان همین نتیجه را دید که دقیقا مطابق انتظار و پاسخ در بخش سوالات است:

```
policy, V= infinite_horizon_policy_iteration(states, actions, transitions, rewards, gamma=gamma_0, epsilon=epsilon)
policy
```

✓ 0.0s

```
0: 'Buy',
1: 'Sell',
2: 'Sell',
3: 'Sell',
4: 'Sell',
5: 'Sell',
6: 'Sell',
7: 'Sell',
8: 'Sell',
9: 'Buy',
10: None}
```

* توجه شود در s=10 اپیزود به پایان میرسید و امکان انجام هیچ کاری نمیباشد.

۲. پیاده سازی به کمک Value Iteration

نتایج در این بخش دقیقا مانند بخش قبل و مطابق انتظار است.

```
policy, V= infinite_horizon_value_iteration(states, actions, transitions, rewards, gamma=gamma_0, epsilon=epsilon)
policy
```

✓ 0.0s

```
{0: 'Buy',
 1: 'Sell',
 2: 'Sell',
 3: 'Sell',
 4: 'Sell',
 5: 'Sell',
 6: 'Sell',
 7: 'Sell',
 8: 'Sell',
 9: 'Buy',
 10: None}
```

- **مقاسه PI و VI:**

هر دو به یک جواب میرسند، باتوجه به یکتا بودن پالیسی در این حالت:

```
print("PI:")
policy_PI, _ = infinite_horizon_policy_iteration(states, actions, transitions, rewards, gamma=gamma_0, epsilon=epsilon)
print(policy_PI)
print("VI:")
policy_VI, _ = infinite_horizon_value_iteration(states, actions, transitions, rewards, gamma=gamma_0, epsilon=epsilon)
print(policy_VI)
```

✓ 0.0s

```
PI:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Sell', 6: 'Sell', 7: 'Sell', 8: 'Sell', 9: 'Buy', 10: None}
VI:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Sell', 6: 'Sell', 7: 'Sell', 8: 'Sell', 9: 'Buy', 10: None}
```

+ Code + Markdown

مقایسه زمان اجرا:

در این بخش مانند حالت قبل هم باتوجه با توابع پیاده‌سازی و ماهیت مسئله و تعداد کم اکشنها میتوان دید که PI به مراتب کندتر از VI بوده است.

```
execution_times_PI = []
execution_times_VI = []

for i in range(10):
    H = np.random.randint(100, 1000)
    execution_time_PI = timeit.timeit(lambda: infinite_horizon_policy_iteration(states, actions, transitions,
    execution_time_VI = timeit.timeit(lambda: infinite_horizon_value_iteration(states, actions, transitions, r
    execution_times_PI.append(execution_time_PI)
    execution_times_VI.append(execution_time_VI)

avg_execution_time_PI = sum(execution_times_PI) / len(execution_times_PI)
avg_execution_time_VI = sum(execution_times_VI) / len(execution_times_VI)

print(f"Avg Execution Time - PI: {avg_execution_time_PI:.5f} seconds")
print(f"Avg Execution Time - VI: {avg_execution_time_VI:.5f} seconds")
```

✓ 0.0s

```
Avg Execution Time - PI: 0.00014 seconds
Avg Execution Time - VI: 0.00002 seconds
```

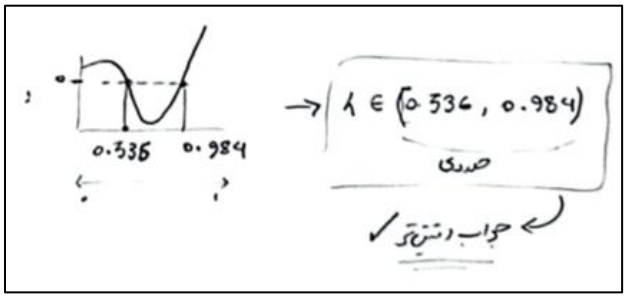
ج)

این بخش نه تنها برای گاما برابر با 0.1 و0.9 برای 0 و 0.5 و 0.55 و 0.999 نیز اجرا میکنیم:

مقایسه و تحلیل:

به صورت تئوری انتظار داریم هرچه گاما بیشتر میشود، agent کمتر به امتیاز لحظه‌ای اهمیت بدهد و بیشتر و در تلاش برای رسیدن به پاداش +۱۰۰ باشد که لازمه‌ی آن انجام عمل "خریدن" به صورت مداوم میباشد، پس انتظار داریم برای 0.9 برای همه و یا بیشتر استیت ها عمل خرید را انتخاب کند تا بتواند در نهایت به امتیاز 100+ برسد و برای 0.1 نیز برعکس است و تنها برای استیت‌های نزدیک به S=10 مانند استیت 9و یا 8 عمل خرید را انجام دهد. (در هرحالتی برای S=0 مجبور به خرید هستیم و این استیت استثنا است.)

به صورت ریاضی و با نگاهی دقیقتر در بخش سوالات نشان دادیم برای گاما بین 0.536 و 0.984 به طور خاص برای استیت s=3، agent عمل فروش را ترجیح میدهد، و پس از مقدار 0.984 ، با بیشتر از ۱۰۰ بار انجام عمل فروش میتوان به مقدار بیشتری پاداش نسبت به زمانی که به S=10 میباشد برسیم. (خلاصه نتایج محاسبات ریاضی در ادامه امده است، اما در بخش سوالات کاملتر به این نکته پرداخته‌ایم.)



در ادامه نتایج را برای تمامی این گاماها به کمک هردو پیاده‌سازی PI و VI نشان میدهیم، که دقیقا مطابق با انتظارات و تحلیل بالا است.

۱. پیاده سازی به کمک Policy Iteration

```
gammas = [0.0, 0.1, 0.5, 0.55, 0.99, 0.999]
for gamma in gammas:
    print(f"gamma = {gamma}:")
    policy, _ = infinite_horizon_policy_iteration(states, actions, transitions, rewards, gamma=gamma, epsilon=epsilon)
    print(policy)
    print("")
```

✓ 0.1s

```
gamma = 0.0:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Sell', 6: 'Sell', 7: 'Sell', 8: 'Sell', 9: 'Buy', 10: None}

gamma = 0.1:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Sell', 6: 'Sell', 7: 'Sell', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.5:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.55:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Buy', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.99:
{0: 'Buy', 1: 'Buy', 2: 'Buy', 3: 'Buy', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.999:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Sell', 6: 'Sell', 7: 'Sell', 8: 'Sell', 9: 'Sell', 10: None}
```

۲. پیاده سازی به کمک Value Iteration

```
gammas = [0.0, 0.1, 0.5, 0.55, 0.9, 0.999]
for gamma in gammas:
    print(f"gamma = {gamma}:")
    policy, _ = infinite_horizon_value_iteration(states, actions, transitions, rewards, gamma=gamma, epsilon=epsilon)
    print(policy)
    print("")
```

✓ 0.0s Python

```
gamma = 0.0:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Sell', 6: 'Sell', 7: 'Sell', 8: 'Sell', 9: 'Buy', 10: None}

gamma = 0.1:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Sell', 6: 'Sell', 7: 'Sell', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.5:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.55:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Buy', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.9:
{0: 'Buy', 1: 'Buy', 2: 'Buy', 3: 'Buy', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.999:
{0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Sell', 6: 'Sell', 7: 'Sell', 8: 'Sell', 9: 'Sell', 10: None}
```

*** نتایج هردو روش دقیقا مطابق انتظار و محاسبات انجام شده است.

- **مقایسه** PI **و** VI:

با توجه به وجود پالیسی بهینه یکتا برای یک گاما مشخص هر دو روش به یک سیاست میرسند و این نتیجه را با اجرای چندبار هردو روش به ازای گاماهای رندوم میتوان دید:

```
for i in range(5):
    gamma = np.random.uniform()
    print(f"gamma = {gamma}:")
    policy_VI, _ = infinite_horizon_value_iteration(states, actions, transitions, rewards, gamma=gamma, epsilon=epsilon)
    policy_PI, _ = infinite_horizon_policy_iteration(states, actions, transitions, rewards, gamma=gamma, epsilon=epsilon)
    if policy_PI == policy_VI:
        print("SAME")
        print("Policy:", policy_PI)
    else:
        print(f"policy_PI = {policy_PI}" )
        print(f"policy_VI = {policy_VI}" )

    print("")
```

✓ 0.0s

```
gamma = 0.360686129038607:
SAME
Policy: {0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Sell', 4: 'Sell', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.7327282774798201:
SAME
Policy: {0: 'Buy', 1: 'Buy', 2: 'Buy', 3: 'Buy', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.5873137696902417:
SAME
Policy: {0: 'Buy', 1: 'Sell', 2: 'Buy', 3: 'Buy', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.527965858576793:
SAME
Policy: {0: 'Buy', 1: 'Sell', 2: 'Sell', 3: 'Buy', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}

gamma = 0.651322721722004:
SAME
Policy: {0: 'Buy', 1: 'Buy', 2: 'Buy', 3: 'Buy', 4: 'Buy', 5: 'Buy', 6: 'Buy', 7: 'Buy', 8: 'Buy', 9: 'Buy', 10: None}
```

مقایسه زمانی:

مجددا مانند بخش "ب" میتوان دید که PI طبق همان تحلیلهای قبلی به مراتب کندتر است:

```
Avg Execution Time - PI: 0.00122 seconds
Avg Execution Time - VI: 0.00010 seconds
```

منابع:

- فایلهای prompt، به صورت PDF، در پوشه LLMs قرار دارد و از GPT کلاس 4o-mini استفاده شده است.

برخی دیگر از منابع:

- https://www.geeksforgeeks.org/implement-value-iteration-in-python/
- https://web.stanford.edu/~lexing/fhopt.pdf
- https://www.youtube.com/watch?v=WsvFL-LjA6U&list=PLoROMvodv4rN4wG6Nk6sNpTEbuOSosZdX&index=1
- https://www.youtube.com/watch?v=gHdsUUGcBC0&list=PLoROMvodv4rN4wG6Nk6sNpTEbuOSosZdX&index=3